Product Catalogue Manager — Comprehensive Application Documentation

1. Executive Summary

Product Catalogue Manager is a fully client-side, single-page web application designed for managing product inventories across four primary textile and fashion categories: **Apparels**, **Fabrics**, **Accessories**, and **Madeups**. The application operates entirely within the browser using HTML5, CSS3, and vanilla JavaScript, with **localStorage** as the persistent data layer. No server, backend, or internet connection is required after the initial file load.

---

2. System Architecture

2.1 Technology Stack

Layer	Technology	Purpose
Structure	HTML5	Semantic markup, form elements, modal dialogs
Styling	CSS3 (Custom Properties, Flexbox, Grid, Animations)	Responsive layout, visual design, transitions
Logic	Vanilla JavaScript (ES6+)	State management, DOM manipulation, event handling
Storage	Web Storage API (localStorage)	Persistent client-side data persistence
File I/O	FileReader API	Image encoding to Base64 for inline storage
Export	Blob API	CSV generation and download

2.2 Architecture Pattern

The application follows a **Model-View-Controller (MVC)** pattern:

- **Model**: JavaScript objects (`items` array) representing product entities
- **View**: HTML/CSS rendering functions that transform model data into DOM elements
- **Controller**: Event listeners and handler functions that bridge user interactions to model updates

2.3 Data Flow

```
User Action → Event Handler → Model Update → Save to localStorage → Re-render View
```

---

3. Data Model

3.1 Product Entity Schema

Field	Type	Required	Description	Validation
`id`	Integer	Auto	Unique identifier (Unix timestamp)	Auto-generated
`name`	String	Yes	Product display name	Non-empty
`category`	Enum	Yes	Product classification	One of: Apparels, Fabrics, Accessories, Madeups
`code`	String	Yes	Stock Keeping Unit (SKU)	Non-empty, user-defined format
`costPrice`	Decimal	Yes	Procurement/ manufacturing cost	0, 2 decimal places
`price`	Decimal	Yes	Selling price to customer	0, 2 decimal places
`mrp`	Decimal	Yes	Maximum Retail Price (strikethrough reference)	0, 2 decimal places
`color`	String	No	Product color name	Free text, auto-mapped to hex for visual dot
`stock`	Integer	Yes	Available inventory quantity	0
`material`	String	No	Fabric/composition details	Free text
`desc`	String	No	Brief product description	Free text
`images`	Array	No	Product image collection	Max 6 images per product

3.2 Image Object Schema

Field	Type	Description
`dataUrl`	String (Base64)	Inline image data for rendering
`filename`	String	Auto-generated SKU-based filename

3.3 Image Naming Convention

The application enforces a strict naming algorithm:

Image Position	Filename Pattern	Example (SKU: YVSM001)
1st (Main)	`{SKU}`	`YVSM001`
2nd	`{SKU}_A`	`YVSM001_A`
3rd	`{SKU}_B`	`YVSM001_B`
4th	`{SKU}_C`	`YVSM001_C`
5th	`{SKU}_D`	`YVSM001_D`
6th	`{SKU}_E`	`YVSM001_E`

Dynamic Re-naming: If the SKU is modified after images are uploaded, all image filenames automatically regenerate to match the new SKU. If images are removed, remaining images are re-sequenced.

---

4. User Interface Components

4.1 Header Section

- **Application Title**: "📦 Product Catalogue Manager"
- **Subtitle**: Category scope descriptor
- **Visual**: Centered layout with bottom border separator

4.2 Category Navigation Tabs

- **"All" Tab**: Displays aggregate count of all products
- **Category Tabs**: Individual tabs for Apparels (👕), Fabrics (🧵), Accessories (👜), Madeups (🛏️)
- **Active State**: Dark blue background (#0f3460) with white text
- **Count Badge**: Dynamic item count per category

4.3 Statistics Bar

Real-time computed metrics displayed as pill-shaped badges:

Statistic	Formula	Purpose
Total Items	`items.length`	Inventory count
Inventory Value	`Σ(price × stock)`	Total selling value
Total Cost	`Σ(costPrice × stock)`	Total procurement cost
Low Stock	`count where stock < 20`	Replenishment alerts
Image Count	`Σ(images.length)`	Media asset tracking

4.4 Search & Controls

- **Search Input**: Real-time filtering across name, SKU, color, material, and description
- **Add Product Button**: Primary CTA (red #e94560) to open creation modal
- **Export CSV Button**: Secondary action to download product data

4.5 Product Card Grid

Responsive CSS Grid (`auto-fill, minmax(300px, 1fr)`) displaying:

Card Components:

1. **Image Area**: 220px height, object-fit cover, hover zoom effect
2. **Placeholder**: Category emoji icon when no images exist
3. **Image Counter Badge**: "+N more" overlay for multi-image products
4. **Product Header**: Name + SKU code badge
5. **Details Section**: Category, Color (with dot), Material, Cost, Price, MRP, Margin %, Stock
6. **Stock Badge**: Color-coded (Green 50, Yellow 20, Red <20)
7. **Action Buttons**: Edit (✏️) and Delete (🗑️)

4.6 Add/Edit Modal

Form Layout:

- Product Name (full width)
- Category + SKU (2-column)
- Cost Price + Selling Price + MRP (3-column)
- Color + Stock Qty (2-column)
- Material (full width)
- Description (textarea, full width)
- Image Upload Area (drag & drop + click)
- Image Preview Grid (3-column, with filenames)

Validation:

- Required fields: Name, Category, SKU, Cost Price, Price, MRP, Stock
- Numeric constraints: Prices 0, Stock 0
- Image limit: Maximum 6 per product

4.7 Lightbox Gallery

- **Overlay**: Dark backdrop (88% opacity) with blur effect
- **Navigation**: Previous/Next arrow buttons + keyboard arrows
- **Counter**: "X / Y" position indicator
- **Filename Display**: SKU-based name at top center
- **Close**: X button, Escape key, or click outside image
- **Animation**: Scale-in effect on open

---

5. Core Functionalities

5.1 CRUD Operations

Create (Add Product)

1. User clicks "+ Add Product" button
2. Modal opens with empty form
3. User fills fields and optionally uploads images
4. On submit: Form data validated → New object created → Pushed to `items` array → Saved to localStorage → View re-rendered → Toast notification shown

Read (Browse & Search)

1. Default view shows all products
2. Category tabs filter by `category` field
3. Search input filters across multiple fields in real-time
4. Product cards display computed fields (margin percentage, stock status)

Update (Edit Product)

1. User clicks "✏️ Edit" on a product card
2. Modal pre-populated with existing data
3. Images loaded with existing filenames preserved
4. On submit: Existing object replaced → Saved to localStorage → View re-rendered

Delete (Remove Product)

1. User clicks "🗑️ Delete" button
2. Browser confirmation dialog appears
3. On confirm: Object removed from array → Saved to localStorage → View re-rendered

5.2 Image Management

Upload Process

```
User selects files → FileReader reads as DataURL → Base64 string stored →
Filename auto-generated from SKU → Preview rendered in grid
```

Drag & Drop Support

- `dragover`: Visual feedback (border color change, background tint)
- `dragleave`: Reset visual state
- `drop`: Process dropped files through same pipeline as file input

Image Constraints

- **Format**: Any browser-supported image type (JPEG, PNG, WebP, GIF, BMP)
- **Size**: Limited by browser localStorage capacity (~5-10MB total)
- **Quantity**: Hard limit of 6 images per product
- **Storage**: Base64 encoded strings (typically 30-50% larger than binary)

5.3 Search Algorithm

```
filtered = items.filter(item => {
  matchesCategory = currentCategory === 'All' || item.category === currentCategory;
  matchesSearch = searchTerm === '' ||
    item.name.toLowerCase().includes(searchTerm) ||
    item.code.toLowerCase().includes(searchTerm) ||
    (item.color && item.color.toLowerCase().includes(searchTerm)) ||
    (item.material && item.material.toLowerCase().includes(searchTerm)) ||
    (item.desc && item.desc.toLowerCase().includes(searchTerm));
  return matchesCategory && matchesSearch;
});
```

5.4 CSV Export

Columns Exported: ID, Name, Category, SKU, Cost Price, Selling Price, MRP, Color, Stock, Material, Description, Image Count

Process:

1. Generate CSV string with headers
2. Iterate items array, escaping quotes and formatting
3. Create Blob with `text/csv` MIME type
4. Generate temporary download link
5. Auto-trigger download with timestamped filename

---

6. Color Detection System

6.1 Color Name Mapping

A comprehensive dictionary maps 80+ common color names to hex codes:

Color Name	Hex Code	Color Name	Hex Code
Red	#e74c3c	Navy Blue	#001f3f
Blue	#3498db	Royal Blue	#4169e1
Green	#2ecc71	Emerald	#50c878
Black	#1a1a1a	Burgundy	#800020
White	#f8f9fa	Champagne	#f7e7ce

6.2 Fallback Algorithm

If no exact match found:

1. Check if input contains any known color substring
2. If still no match, generate deterministic RGB from string hash
3. Display as inline `rgb()` value

6.3 Visual Representation

- 16px circular dot displayed before color name text
- 2px light gray border for visibility
- Subtle box shadow for depth

---

7. Business Logic

7.1 Margin Calculation

```
Margin % = ((MRP - Cost Price) / MRP) × 100
```

Displayed to 1 decimal place on each product card.

7.2 Stock Status Classification

Status	Condition	Visual
High	stock 50	Green badge
Medium	20 stock $<$ 50	Yellow/amber badge
Low	stock $<$ 20	Red badge

7.3 Price Hierarchy Enforcement

The UI implies: **Cost Price Selling Price MRP** (Note: Validation enforces numeric non-negativity but does not strictly enforce the hierarchy — this is left to business process.)

---

8. Responsive Design

8.1 Breakpoints

Range	Layout Adjustments
$>$ 768px	Multi-column grid, side-by-side form fields
768px	Single column grid, stacked form fields
480px	Full-width controls, reduced padding

8.2 Mobile Optimizations

- Touch-friendly button sizes (min 44px tap target)
- Modal max-height: 92vh with internal scroll
- Lightbox navigation buttons enlarged to 52px
- Form fields stack vertically on narrow screens

---

9. Browser Compatibility

Browser	Minimum Version	Notes
Chrome	60+	Full support
Firefox	60+	Full support
Safari	12+	Full support
Edge	79+	Full support (Chromium-based)
IE	Not supported	ES6+ features, CSS Grid

9.1 Required APIs

- `localStorage` (Web Storage)
- `FileReader` (File API)
- `Blob` / `URL.createObjectURL` (File API)
- CSS Grid & Flexbox
- CSS Custom Properties (optional fallback available)

---

10. Data Persistence & Security

10.1 localStorage Structure

```
Key: "catalogueItemsV3"
Value: JSON stringified array of product objects
```

10.2 Storage Considerations

- **Capacity**: ~5-10MB depending on browser
- **Images**: Base64 encoding increases size by ~33%
- **Recommendation**: For large image catalogs, consider external storage or image compression
- **Privacy**: Data never leaves the browser; no server communication
- **Backup**: Users should periodically export CSV as backup

10.3 Data Migration

If upgrading from previous versions:

- V1 data key: `catalogueItems`
- V2 data key: `catalogueItemsV2`
- V3 data key: `catalogueItemsV3`
- Manual migration or data re-entry required between major versions

---

11. User Workflows

11.1 Adding a New Product

1. Click "+ Add Product"
2. Enter product name
3. Select category from dropdown
4. Enter SKU code (e.g., YVSM001)
5. Enter Cost Price, Selling Price, and MRP
6. Enter color name (e.g., "Navy Blue")
7. Enter stock quantity
8. Enter material and description (optional)
9. Upload images (optional) — filenames auto-generate from SKU
10. Click "Save Product"
11. Toast confirms success; card appears in grid

11.2 Editing a Product

1. Locate product in grid (use search or tabs)
2. Click "✏️ Edit"
3. Modify desired fields
4. Add/remove images as needed
5. Click "Save Product"

11.3 Exporting Data

1. Click "📩 Export CSV" button
2. File auto-downloads as `product_catalogue_YYYY-MM-DD.csv`
3. Open in Excel, Google Sheets, or any CSV viewer

---

12. File Structure

```
catalogue_app.html          # Single self-contained file
├── <head>
│   ├── Meta tags (charset, viewport)
│   ├── <style>             # All CSS (no external dependencies)
│   └── Google Fonts (Inter, system fallback)
├── <body>
│   ├── .catalogue-container
│   │   ├── .header
│   │   ├── .tabs
│   │   ├── .stats-bar
│   │   ├── .controls
│   │   ├── .items-grid
│   │   └── .empty-state
│   ├── .modal-overlay      # Add/Edit modal
│   ├── .lightbox-overlay   # Image gallery
│   ├── .toast             # Notification system
│   └── <script>           # All JavaScript
│       ├── Data & Config   # categories, colorMap, demo data
│       ├── State Variables # items, currentCategory, tempImages
│       ├── Core Functions  # saveItems, renderTabs, renderStats
│       ├── View Functions  # renderItems, renderImagePreviewGrid
│       ├── Modal Functions # openModal, closeModal, openLightbox
│       ├── Event Handlers  # CRUD, search, upload, export
│       └── Initialization  # renderTabs, renderStats, renderItems
```

---

13. Performance Characteristics

Metric	Target	Notes
First Paint	< 500ms	Single file, no external requests
Search Response	< 50ms	Client-side filtering, small dataset
Modal Open	< 100ms	Pre-rendered DOM, CSS animation
Image Upload	< 2s	Depends on file size and device
localStorage Save	< 50ms	JSON stringify + write
Supported Dataset	~500 products	Limited by localStorage capacity

---

14. Known Limitations

1. **No Multi-User Support**: Single-user, single-device usage
2. **No Cloud Sync**: Data exists only in browser localStorage
3. **Image Size**: Large images may exceed storage quotas
4. **No Undo**: Delete operations are permanent (with confirmation)
5. **No Batch Operations**: Products must be edited individually
6. **No Print Styles**: Optimized for screen viewing only

---

15. Future Enhancement Roadmap

Priority	Feature	Complexity
High	Image compression before storage	Medium
High	Import CSV functionality	Medium
Medium	Sortable columns (price, stock, name)	Low
Medium	Bulk delete / multi-select	Medium
Medium	Print-friendly stylesheet	Low
Low	Dark mode toggle	Low
Low	Barcode/QR code generation from SKU	Medium
Low	Service Worker for offline PWA	High

---

16. Prompt for AI Reproduction

```
Create a single-file HTML web application for managing a product catalogue with the following
↪  requirements:

CORE REQUIREMENTS:
1. Four product categories: Apparels, Fabrics, Accessories, Madeups
2. Product fields: Name, SKU/Code, Cost Price, Selling Price, MRP, Color, Stock Quantity, Material,
↪  Description
3. Image upload support (max 6 per product) with drag-and-drop
4. Auto-generated image filenames based on SKU: Main=SKU, Additional=SKU_A, SKU_B, SKU_C, etc.
5. Dynamic filename re-generation when SKU changes
6. Visual color dot that auto-detects 80+ common color names and maps to hex codes
7. Real-time search across name, SKU, color, material, and description
8. Category tab filtering with item counts
9. Statistics bar showing: total items, inventory value, total cost, low stock count, image count
10. Stock status badges: Green (≥50), Yellow (≥20), Red (<20)
11. Margin percentage calculation: ((MRP - Cost) / MRP) × 100
12. Lightbox image gallery with navigation, keyboard controls, and filename display
13. CSV export functionality
14. Toast notifications for user feedback
15. All data persisted in browser localStorage
```

```
16. Fully responsive design (mobile + desktop)
17. No external dependencies (single HTML file)

TECHNICAL SPECIFICATIONS:
- Use vanilla JavaScript (ES6+) with no frameworks
- Use CSS Grid and Flexbox for layouts
- Use CSS transitions and keyframe animations for smooth UX
- Implement MVC pattern with clear separation of concerns
- Handle edge cases: empty states, no search results, storage limits
- Include 8 demo products with realistic data
- Support keyboard navigation (Escape, Arrow keys)
- Include print styles or note lack thereof

DESIGN SPECIFICATIONS:
- Color palette: Primary #0f3460 (navy), Accent #e94560 (coral red), Background #f5f7fa
- Card-based product grid with hover effects
- Modal with form validation and image preview grid
- Clean, modern typography using Inter font family
- Smooth animations on modal open, card hover, and lightbox

DELIVERABLE:
A single, self-contained HTML file that opens in any modern browser and functions as a complete product
 ↪  catalogue management system.
```

Document Version: 1.0
Generated: 2026-07-07
Application Version: v3.0